Assignment 1: Environment Setup and Requirement Gathering

Title

Environment Setup and Requirement Gathering for Full Stack Web Application **(**Computer Institute Management System)

Aim / Objective

To install required development tools, select appropriate frontend and backend technologies, and gather requirements for developing a full stack web application. **(**Computer Institute Management System)

Problem Statement

Before developing any web application, it is important to set up a proper development environment and understand the client's requirements. This practical focuses on installing necessary tools, choosing suitable technologies, and identifying user needs for building an efficient and scalable web application.

Theory / Concept

Full Stack Development involves working on both:

- **Frontend (Client-side):** User interface and user experience

- **Backend (Server-side):** Logic, database, and APIs

Tools Used

- **VS Code:** Code editor
- **Git:** Version control system
- **Postman:** API testing tool
- **Node.js:** Runtime environment

Technology Selection

- **Frontend Options:**
    - Vanilla HTML, CSS, JavaScript
    - React.js

- **Backend Options:**
    - Node.js with Express
    - MangoDB

Tools & Technologies Used

- Visual Studio Code
- Git & GitHub
- Postman
- Node.js
- React.js
- Express.js

Steps / Procedure

Step 1: Install Development Tools

- Installed VS Code
- Installed Git and configured it
- Installed Node.js
- Installed Postman

Step 2: Setup Project Environment

- Created project folder
- Initialized project using Node.js
- Verified installations using version commands

Step 3: Select Frontend Technology

- Selected **HTML,CSS** , **React.js** for building dynamic and responsive UI

Step 4: Select Backend Technology

- Selected **Node.js with Express** for server-side development

Step 5: Identify Client / Organization

- The selected client for this project is a **Computer Training Institute**, namely **"RC Rudraksha Computers"**.

Step 6: Requirement Gathering

Collected requirements using:

- Discussion with client
- Understanding user needs

Functional Requirements:

- User registration and login
- Add / manage students, courses
- Add / manage students

- Track student attendance
- Generate reports
- View student progress

Non-Functional Requirements:

- User-friendly interface
- Fast performance
- Secure data handling

Expected Output / Result

- Development environment successfully set up
- Required tools installed and configured
- Technologies selected for development
- Client requirements documented clearly

Conclusion

This practical helped in understanding the importance of environment setup and requirement gathering before starting any project. It ensures proper planning, technology selection, and smooth development process.

Front-end Technology

Technology	Role	Key Features	Limitations	Best Use
HTML, CSS, JavaScript (Vanilla JS)	Core technologies for building web interfaces	- Full control over structure (HTML), styling (CSS), and logic (JS) - No dependencies or frameworks required - Lightweight and fast for small projects	- Code becomes difficult to manage in large applications - No built-in state management - Reusability is limited (no components)	Static websites, small projects, learning fundamentals
React.js	Component-based frontend library for building dynamic UIs	- Reusable components → improves scalability - Virtual DOM → fast rendering and performance optimization - Strong ecosystem (Redux, React Router) - Supports SPA (Single Page Applications) - Easy API integration	- Initial setup complexity - Requires understanding of JSX and hooks - Additional libraries needed for full features	Large-scale applications, dashboards, admin panels, real-time apps
Vue.js	Progressive JavaScript framework for UI development	- Simple and easy to learn - Two-way data binding - Lightweight and flexible - Can be integrated into existing projects easily	- Smaller community compared to React - Limited enterprise-level support - Fewer third-party libraries	Medium-sized applications, quick development projects
Angular	Full-fledged frontend framework by Google	- Complete MVC architecture - Built-in tools (routing, forms, HTTP client) - TypeScript support (strong typing) - High scalability for enterprise apps	- Steep learning curve - Heavy framework (larger bundle size) - Overkill for small projects	Enterprise-level applications, large-scale systems

Back-end technology:

Technology	Role	Key Features	Limitations	Best Use
Node.js + Express	Backend (Server-side)	Fast, non-blocking I/O, same language (JavaScript), scalable APIs	Callback complexity (handled using async/await)	Real-time apps, REST APIs
Django (Python)	Backend framework	Built-in security, admin panel, ORM support	Slower for real-time apps, Python dependency	Secure enterprise applications
PHP	Backend scripting	Easy hosting, widely supported	Less modern, harder to scale	Small to medium web apps
MongoDB	NoSQL Database	Flexible schema, JSON-like data, easy integration with Node.js	No strict schema, less relational support	Dynamic data (students, courses)
MySQL	Relational Database	Structured data, reliable, ACID compliant	Less flexible schema	Banking, structured data systems
PostgreSQL	Advanced SQL Database	High performance, advanced queries, secure	Slightly complex setup	Enterprise-level applications

Justification

Frontend– React.js

React.js is selected as the frontend technology for the following reasons:

- **Component-Based Architecture**
  The system requires multiple UI sections such as **student dashboard, admin panel, course management, and reports**. React allows building reusable components, making the code modular and maintainable.
- **Efficient Rendering (Virtual DOM)**
  Features like attendance updates, student data changes, and dashboard refresh require fast UI updates. React's Virtual DOM ensures high performance and smooth user experience.
- **Scalability**
  The project can grow in future (adding features like online tests, notifications). React supports scalable architecture, making it suitable for long-term use.

Backend– Node.js with Express

- **Single Language (JavaScript Everywhere)**
  Both frontend and backend use JavaScript, reducing complexity and improving development efficiency.
- **Fast & Non-Blocking Architecture**
  Node.js uses an event-driven, non-blocking I/O model, making it highly efficient for handling multiple requests like login, attendance updates, and data fetching.
- **Lightweight Framework (Express.js)**
  Express provides a simple and flexible structure for building APIs required for:
  - Student management
  - Course handling
  - Authentication